

Lectura 2: Database Caching Strategies Using Redis

Bases de datos II

Estudiante: Fabricio Monge Brenes Carnet: 2023108707

Instrucciones:

Tiempo estimado de lectura: 30 minutos

Preguntas:

1. Explique los desafíos que se enfrentan en la construcción de sistemas distribuidos que requieren baja latencia (de acuerdo con la lectura)
2. Explique los tipos de caching, ¿Cuál considera es el más adecuado?
3. Explique las diferencias entre los patrones de caching.
4. Explique que es cache eviction, ¿Por qué es relevante para el rendimiento de un sistema?

Respuestas:

1.
 - Queries procesados lentamente: el obtener datos de del disco y el procesamiento del query añadido pueden llevar a los tiempos de respuesta a velocidades de más de dos dígitos en milisegundos.
 - Cost to scale: el escalado de para grandes cantidades de lecturas puede ser costoso así sea una base de datos NoSQL o una base de datos relacional. se puede llegar a necesitar de varias replicas de bases de datos de lectura para compararlo con una solo nodo de cache en memoria.
 - The need to simplify data access: las bases de datos relacionales son buenas para la relación entre modelos de datos, pero no son optimas para el acceso a los datos. Hay instancias donde una app puede buscar acceder a los datos en una estrucura especifica para simplificar la obtención de datos e incrementar el desempeño de la app
- 2.

- Cache integrado en base de datos: algunas bd como Amazon Aurora ofrecen un cache integrado que se maneja desde el mismo motor de la base de datos y cuentan con capacidad de Write-through. La base de datos actualiza e cache automaticamente cuando los datos cambian. El problema con estas es que no estos es que pueden ser limitados en tamaño y en capacidades
- Caches locales: un cache local guarda los datos más utilizados dentro de la app, generando recoleccion de datos más rápido que con oreas arquitecturas de cache pues no hay que depender de una red. El problema principal es que los nodos de cache local se encuentran desconectados entre si, por lo que se vuelve complicado en un ambiente distribuido dónde se necesite de compartir información constantemente. Además si una app utiliza varios servers entonces se vuelve dificil coordinar los valores entre estos y si ocurre un apagón los datos se pueden perder.
- Caches remotos: llamados también "side cache" son instancias separadas dedicadas a guardar datos en memoria para utilización como cache, se guardan en servers dedicados y normalmente se crean como Key-value NoSQL dbs como Redis o Memcached. Permiten hasta 1 millon de requests por segundo por cada nodo de cache. Cuentan con respuestas en sub milisegundos por lo que son más rapidas que una db basada en disco.
- El tipo de caching más adecuado depende mucho de la app a desarrollar, pero, si se basa en latencia, un cache remoto es la mejor opción, pues ofrece respuesatas rápidas y puede trabajar en paralelo con otras bases de datos u otros sistemas de cache (puesto que la lectura menciona que se puede trabajar con cache remoto y otros modelos de cache con un sistema de tiers).

3.

- Existen dos patrones de caching, el Cache-Aside o Lazy Loading que se basa en cargar los datos a la cache después de haberse obtenido. Por lo que esta primero hace consulta en la cache para revisar si se encuentran los datos y en caso de que se haga un "cache miss" se procede a consultar la base de datos para obtener la información, entregarla al usuario y añadirla al cache. Por otro lado el patrón Write-through se basa en escribir los datos en cache cuando la base de datos es actualizada por lo que contiene información reciente.
- Como se puede evidenciar, la lógica principal de estas caches es distinta. Además la Write-through cache puede volverse menos eficiente en terminos de uso de espacio pues puede tener datos poco consultados a diferencia de que una Cache-aside cuenta únicamente con la información más consultada. Sin embargo, ya que esta última se actualiza por cada cache miss, es menos ágil en dar las resupuestas pues necesita de lectura en BD.

4.

- El cache eviction es el proceso de tomar algunas keys de la cache y eliminarlos cuando la cache se llega a llenar o pasa del parametro "maxmemory". Las bases de datos utilizan "políticas" de evicción? entre las cuales se basan en LRU ("least recently used") o LFU ("least frequently used"). Algunas configuraciones de políticas de evicción se basan en eliminar keys sin importar el TTL (tiempo de vida o "time to live"), eliminar keys que cuentan con TTL definido, eliminar las cuales tengan el TTL más corto, eliminar aleatoreamente de entre las que tienen TTL o en general, y la opción de desactivar el cache eviction.
- Este proceso es importante pues permite que el sistema tenga espacio para llenar la cache con nuevos datos y así los datos se puedan acceder más rápidamente. Además, cuando se llega a tener este sistema también se puede tomar como un indicio a escalar la cache. El escalado puede ser hacia arriba,

añadiendo más memoria, o hacia fuera, añadiendo más nodos de cache. Otro aspecto es que, el LFU permite que solo se mantenga en la cache los datos más consultados por lo que estos pueden seguir estando a una rápida disposición y dando respuestas rápidas, incluso después de que se ejecute el cache eviction.